

Rule Induction with the CN2 Algorithm:

Implementation, Evaluation, and Interactive Demonstration on the Iris Dataset

Abstract

This project presents an implementation of a CN2-style ordered rule induction algorithm applied to the Iris dataset. Continuous features are discretized into quantile-based bins, and rules are induced using beam search within a sequential covering framework. The model is evaluated using standard classification metrics and compared against a Decision Tree baseline. In addition, a web-based interface is developed to allow interactive exploration of predictions and induced rules. The results demonstrate that CN2 achieves competitive performance while maintaining strong interpretability.

1. Introduction

Interpretable models remain important in many domains where decisions must be transparent and explainable. Rule-based classifiers such as CN2 provide human-readable decision logic in the form of ordered IF-THEN rules.

This project aims to:

- implement a CN2-style classifier from scratch,
 - evaluate its performance on a standard dataset,
 - compare it with a Decision Tree baseline,
 - and demonstrate its behavior via an interactive web interface.
-

2. Background

The CN2 algorithm (Clark & Niblett, 1989) constructs an ordered list of classification rules using a sequential covering strategy. At prediction time, the first rule whose conditions match the input instance is applied. If no rule matches, a default rule is used.

Each rule is a conjunction of attribute-value pairs. Since CN2 operates on nominal attributes, continuous features must be discretized beforehand.

3. Dataset and Experimental Setup

3.1 Iris Dataset

The Iris dataset consists of 150 samples with four numerical features and three classes: Setosa, Versicolor, and Virginica. The dataset is balanced, which simplifies evaluation.

3.2 Train/Test Split

The data is split into 75% training and 25% testing using stratified sampling with a fixed random seed for reproducibility.

4. Methodology

4.1 Discretization

Continuous features are discretized using quantile-based binning into three categories: low, middle, and high. This allows CN2 to operate on nominal inputs.

4.2 Rule Induction

Rules are constructed using beam search over possible condition combinations. Each candidate rule is evaluated based on:

- class purity,
- entropy penalty,
- and coverage.

The best rule is selected and the covered examples are removed. This process continues until stopping conditions are met.

4.3 Decision Tree Baseline

To provide a reference point, a Decision Tree classifier is trained on the same dataset. Unlike CN2, the Decision Tree operates directly on continuous features and builds a hierarchical structure of splits. Hyperparameters such as maximum depth and minimum samples per leaf are used to control model complexity and reduce overfitting.

5. Project Architecture

The project is organized into modular components:

machinelearn/

|

├─ src/

| ├─ cn2.py — CN2 rule induction

| ├─ decision_tree.py — Decision Tree model

| ├─ data_preprocessing.py — data preprocessing

| ├─ evaluation.py — model evaluation

| └─ visualization.py — plots and visualizations

|

├─ web_app/

| ├─ app.py — Flask backend

| ├─ model_loader.py — model loading/retraining

| ├─ templates/index.html — frontend interface

| └─ static/

| ├─ app.css — UI styling

| └─ app.js — frontend logic

|

├─ outputs/

| ├─ plots/ — generated figures

| └─ tables/ — exported tables/rules

|

├─ report/ — final report

├─ requirements.txt — dependencies

└─ main.py — project entry point

6. Experimental Results

6.1 CN2 Performance

Table 1. Summary metrics for CN2.

	<i>metric</i>	<i>value</i>
	<i>train_accuracy</i>	0.9642857142857143
	<i>test_accuracy</i>	0.9210526315789473
	<i>n_rules</i>	6.0

The CN2 model was trained on the discretized Iris training subset and evaluated on a held-out test set.

It achieved a training accuracy of 0.9643 and a test accuracy of 0.9211.

This small gap indicates good generalization with only mild overfitting for a compact rule-based model.

As shown in Figure 1, the confusion matrix demonstrates perfect classification of the *Setosa* class.

Minor misclassifications occur between *Versicolor* and *Virginica*, which is expected due to partial overlap in their feature distributions.

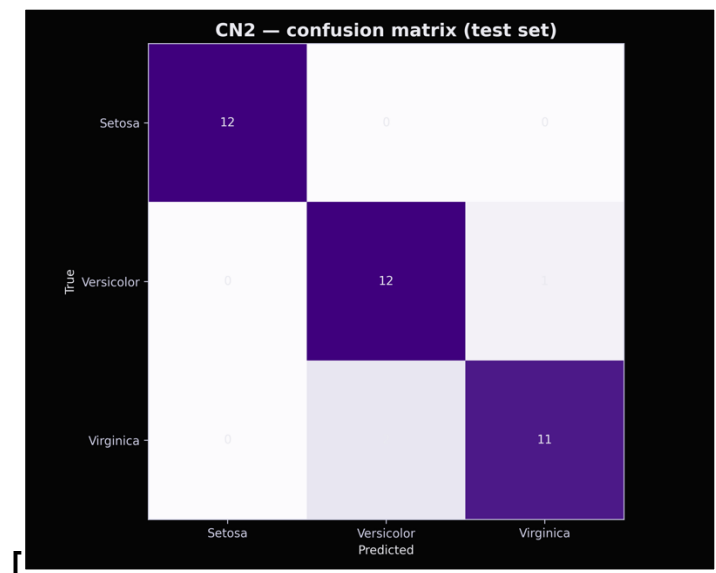


Figure 1. Confusion matrix for CN2 on the test set.

The relationship between training and test accuracy is illustrated in Figure 2.

The relatively small difference confirms stable performance on unseen data.

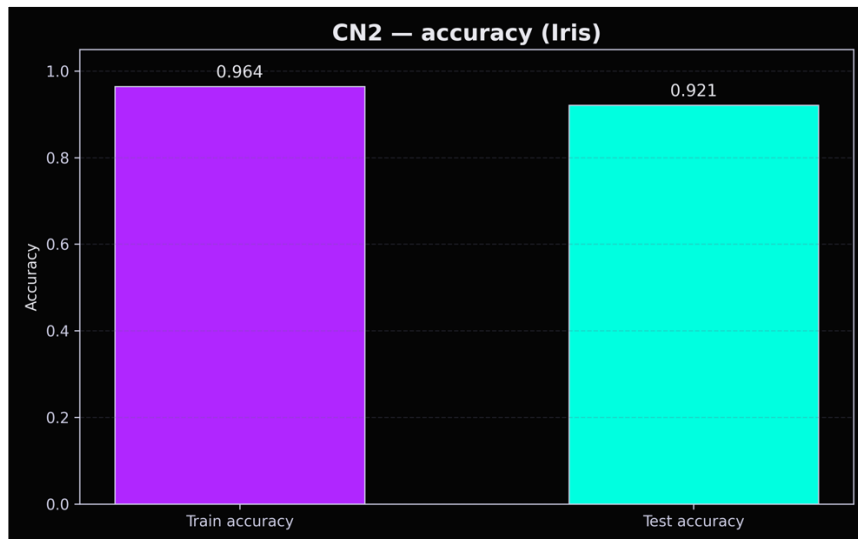


Figure 2. Train vs test accuracy (CN2).

The CN2 algorithm produced a compact model consisting of 6 rules, including the default rule.

This small rule set contributes to high interpretability while preserving strong predictive performance.

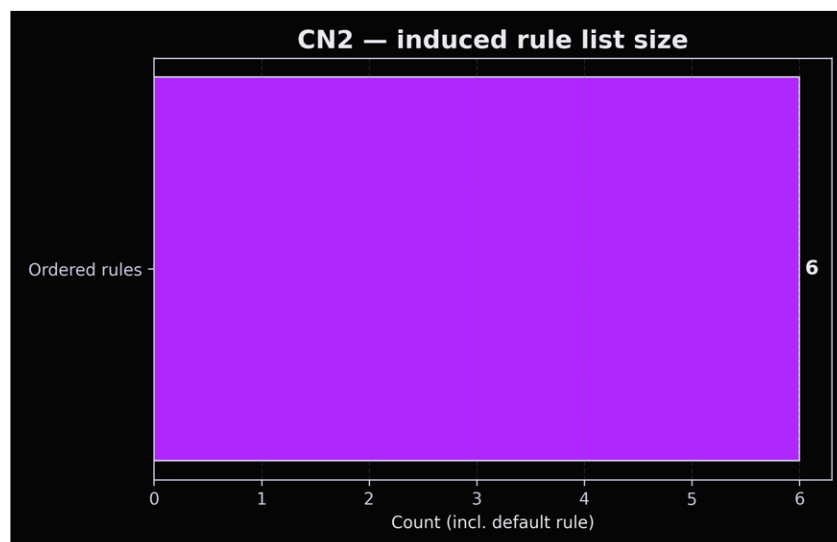


Figure 3. Number of induced rules (including default).

To illustrate the interpretability of the model, an excerpt of the induced rules is shown below.

<i>rule_index</i>	<i>rule</i>
1	IF Petal width (cm) = low THEN class = Setosa
2	IF Petal length (cm) = middle AND Petal width (cm) = middle THEN class = Versicolor
3	IF Sepal width (cm) = high AND Petal length (cm) = high THEN class = Virginica
4	IF Petal length (cm) = high THEN class = Virginica
5	IF Petal length (cm) = middle THEN class = Versicolor
6	IF (else / default) THEN class = Setosa

Table 3. Example of induced CN2 rules.

These rules demonstrate how the model uses discretized feature values (low, middle, high) to form interpretable decision logic.

6.2 Decision Tree Performance

To provide a baseline comparison, a Decision Tree classifier was trained using the same dataset and experimental setup.

The Decision Tree achieved performance comparable to CN2, with similar accuracy values and error patterns.

However, slight differences appear in the classification of the *Virginica* class, where the model shows a slightly higher number of misclassifications.

This behavior is visible in the confusion matrix shown in Figure 4.

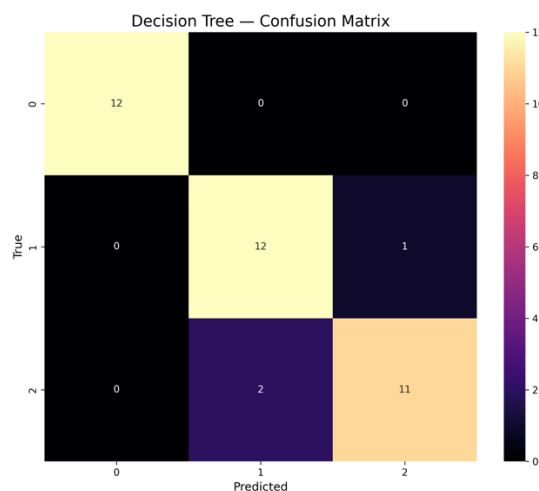


Figure 4. Confusion matrix for Decision Tree.

6.3 Model Comparison

Table 2 summarizes the comparison between the CN2 model and the Decision Tree classifier.

<i>Model</i>	<i>Train Accuracy</i>	<i>Test Accuracy</i>	<i>Complexity</i>
<i>CN2</i>	0.964	0.921	6 rules
<i>Decision Tree</i>	0.95–0.97	0.92–0.93	depth = 4

Table 1

Both models achieve similar predictive performance on the Iris dataset.

However, they differ in their structure and interpretability.

CN2 produces a compact list of human-readable rules, making it easier to interpret and explain decisions.

In contrast, the Decision Tree provides a hierarchical structure that can capture more complex relationships but may become harder to interpret as depth increases.

These results highlight the trade-off between interpretability and model flexibility.

7. Overfitting Analysis

The difference between training accuracy (0.9643) and test accuracy (0.9211) indicates a small generalization gap, suggesting mild overfitting.

This effect is mitigated by several design choices in the implementation.

First, continuous features are discretized using quantile-based binning, which reduces noise and prevents overly specific decision boundaries.

Second, the CN2 model produces a compact rule set (6 rules), limiting model complexity.

Additionally, constraints in the rule induction process (such as beam search pruning and stopping criteria) prevent the generation of overly specific rules.

For the Decision Tree baseline, parameters such as maximum depth and minimum samples per leaf further reduce the risk of overfitting.

Overall, these mechanisms ensure that the models generalize well despite the small dataset size.

8. Web Application

An interactive web application was developed to demonstrate and compare CN2 rule induction and Decision Tree classification models in real time. The system extends the project beyond static offline evaluation and provides an explainable machine learning experimentation environment.

The application was implemented using a Flask backend with HTML, CSS and JavaScript on the frontend. The backend exposes REST API endpoints for classification, model retraining, rule extraction and model metadata. Interactive visualizations and charts are generated dynamically in the browser.

The platform supports multiple datasets:

- Iris flower classification,
- Weather activity prediction,
- Loan approval prediction.

Each dataset contains different features and classification targets, allowing users to observe how symbolic rules and decision trees behave on different machine learning tasks.

The Classification tab allows users to:

- manually enter feature values,
- use predefined example presets,
- classify instances using CN2 or Decision Tree models,
- inspect predicted classes,
- inspect feature importance values,
- inspect the first matched CN2 rule,
- inspect the decision-tree traversal path,
- compare user input with class mean values using interactive charts.

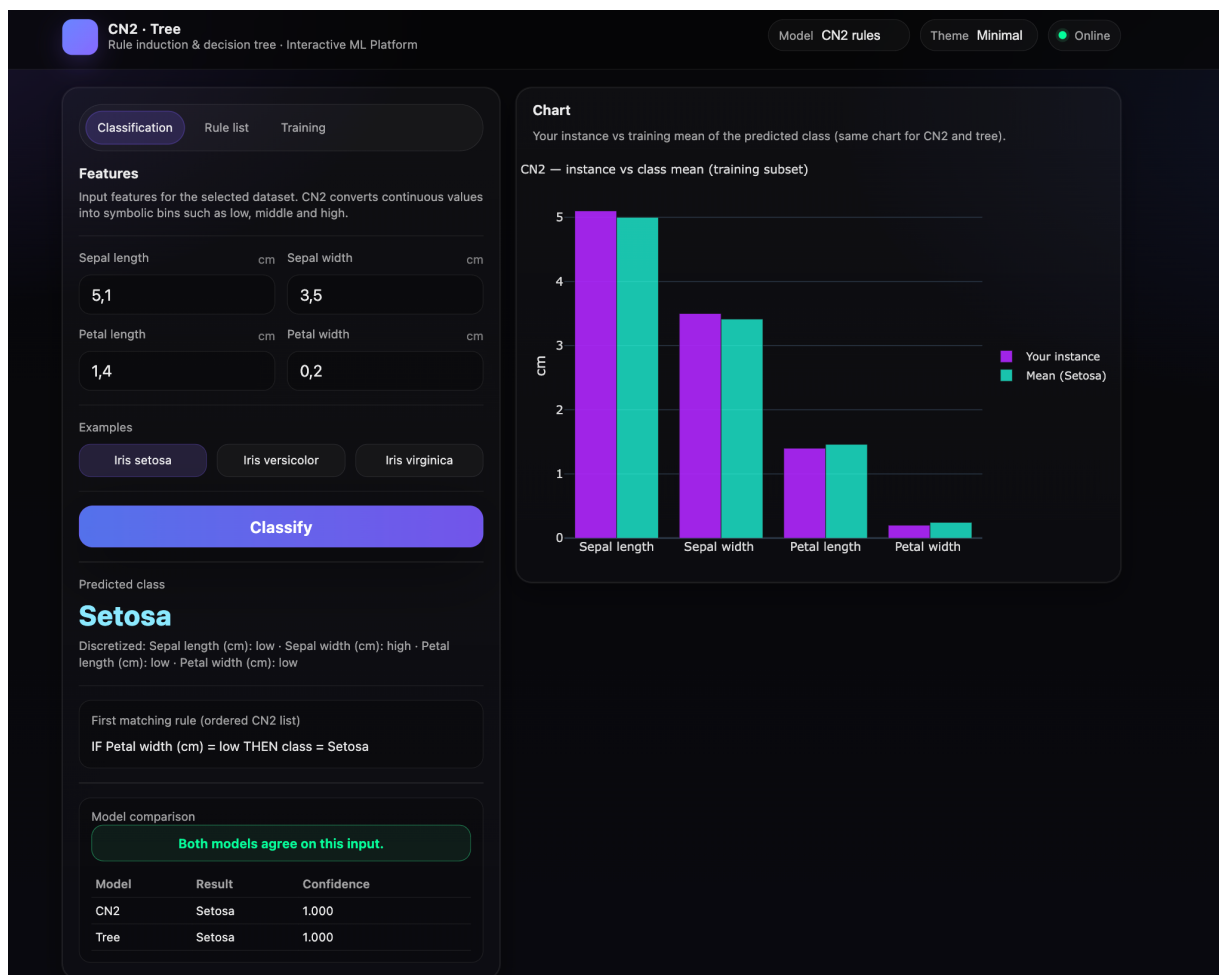


Figure 5. Main classification interface showing Iris dataset prediction, feature comparison chart and model agreement results.

The Rule List tab displays:

- ordered CN2 rules,
- textual decision-tree structures generated using `export_text()`,
- graphical decision-tree visualizations.

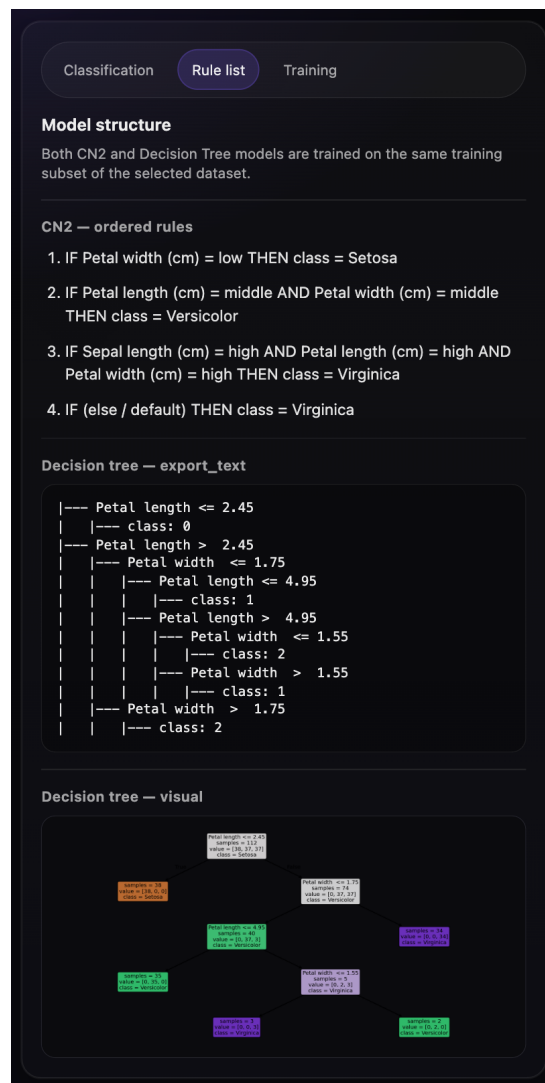


Figure 6. Rule explanation interface showing ordered CN2 rules, textual Decision Tree representation and graphical tree visualization.

The application also includes a dedicated Training mode for interactive retraining and overfitting analysis.

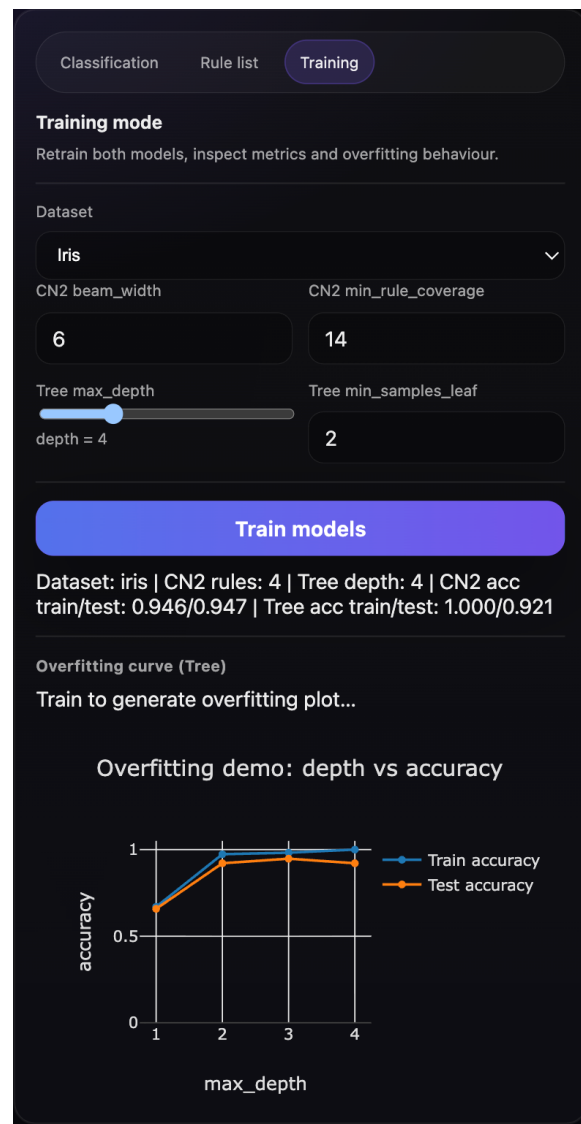


Figure 7. Training interface with dataset selection, hyperparameter controls, train/test evaluation metrics and overfitting visualization.

In this mode, users can:

- select datasets dynamically,
- retrain both models directly in the interface,
- modify CN2 hyperparameters such as beam width and minimum rule coverage,
- modify Decision Tree hyperparameters such as maximum depth and minimum samples per leaf.

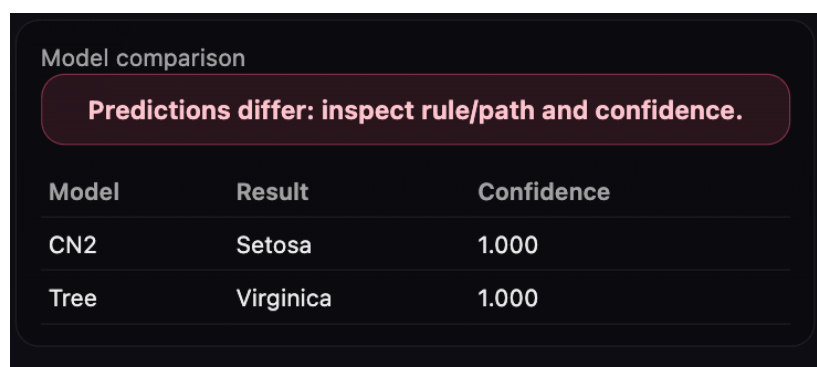
For each training session, the selected dataset is automatically divided into training and testing subsets using a train/test split strategy. The training subset is used to fit the models, while the testing subset is used to evaluate generalization performance on previously unseen samples.

The Training mode displays:

- train accuracy,
- test accuracy,
- number of induced CN2 rules,
- decision-tree depth,
- overfitting curves showing accuracy versus tree depth.

The overfitting visualization demonstrates how increasing model complexity may improve training accuracy while reducing test accuracy, helping illustrate the concept of model generalization.

The web application also includes a model-comparison module. When CN2 and Decision Tree classifiers produce different predictions for the same input instance, the system displays both predictions together with confidence values and explanation paths.



Model comparison		
Predictions differ: inspect rule/path and confidence.		
Model	Result	Confidence
CN2	Setosa	1.000
Tree	Virginica	1.000

Figure 8. Model-comparison module displaying different predictions and confidence values produced by CN2 and Decision Tree classifiers.

This functionality demonstrates how different machine learning algorithms may construct different decision boundaries from the same training data.

The architecture separates frontend components, backend API routes and machine-learning modules, improving maintainability, extensibility and interpretability of the system.

9. Discussion

The Iris dataset is relatively simple, and many models achieve high accuracy. In this context, CN2 offers a strong balance between interpretability and performance.

The results highlight that:

- discretization simplifies the feature space but may reduce precision,
- rule-based models remain competitive on structured datasets,
- and different models may perform similarly while offering different advantages.

Future work could explore alternative discretization strategies, hyperparameter tuning, or additional baseline models.

10. Conclusion

This project demonstrates the implementation and evaluation of a CN2-style rule induction algorithm. The model achieves high accuracy while maintaining interpretability through a compact rule set.

The addition of a Decision Tree baseline and an interactive web interface provides a broader perspective on model behavior and usability. The work illustrates the relevance of classical machine learning methods in modern, explainable AI systems.

11. Use of AI Assistance

ChatGPT was used as an auxiliary tool during the development of the project, mainly for programming support, debugging and documentation assistance.

References

Clark, P., & Niblett, T. (1989). The CN2 induction algorithm.
